

DOCUMENTACIÓN TÉCNICA

PROYECTO KRONOLEON

Sistema de Gestión de Guardias

Fecha: 14 de diciembre de 2025

Backend - Django 6.0

ÍNDICE

1. Estructura General del Proyecto
2. Aplicaciones Django
3. Configuración Principal
4. Flujo de Autenticación
5. Modelos y Base de Datos
6. Dependencias y Requisitos
7. Rutas y Endpoints
8. Seguridad y Validación
9. Guía de Desarrollo

1. ESTRUCTURA GENERAL DEL PROYECTO

El proyecto **Kronoleon** es un Sistema de Gestión de Guardias para profesores construido con **Django 6.0** como framework backend y **MySQL** como base de datos. El sistema permite a los profesores visualizar sus guardias asignadas, consultar horarios, y gestionar su agenda de forma eficiente.

Estructura de directorios:

```
kronoleon/
■ ■ ■ kronoleon/ # Configuración central del proyecto Django
■ ■ ■ ■ settings.py # Configuración del proyecto
■ ■ ■ ■ urls.py # Enrutamiento URL principal
■ ■ ■ ■ wsgi.py # Configuración WSGI
■ ■ ■ ■ asgi.py # Configuración ASGI
■ ■ ■ usuarios/ # App de autenticación
■ ■ ■ ■ templates/ # Plantillas HTML
■ ■ ■ ■ models.py # Modelos de datos
■ ■ ■ ■ views.py # Lógica de vistas
■ ■ ■ ■ urls.py # URLs de la app
■ ■ ■ dashboard/ # App principal del dashboard
■ ■ ■ ■ templates/ # Plantillas HTML
■ ■ ■ ■ models.py # Modelos de datos
■ ■ ■ ■ views.py # Lógica de vistas
■ ■ ■ ■ urls.py # URLs de la app
■ ■ ■ manage.py # Script de gestión Django
```

2. APLICACIONES DJANGO

2.1. Aplicación: usuarios

Propósito: Gestionar la autenticación y login de usuarios (profesores).

Rutas definidas:

- GET/POST /login/ → Vista de login
- GET /logout/ → Vista de logout

Componentes principales:

- **login_view(request):** Maneja la autenticación mediante AuthenticationForm de Django. Valida credenciales, redirige al dashboard si el usuario ya está autenticado, y muestra mensajes de error si la autenticación falla.
- **Plantilla login.html:** Diseño responsive con gradiente azul-púrpura, formulario de login con validación integrada y mensajes de error personalizados.

2.2. Aplicación: dashboard

Propósito: Panel principal donde los profesores ven sus guardias asignadas, estadísticas y agenda.

Rutas definidas:

- GET /home/ → Vista de inicio (redirige a dashboard)
- GET /dashboard/ → Vista principal del dashboard

Modelos definidos:

Modelo	Campos Principales	Tabla BD
Profesor	nombre, apellido1, apellido2, email, activo	PROFESOR
Asignatura	nombre, activa	ASIGNATURA
Aula	nombre (único)	AULA
HoraGuardia	dia, hora_inicio, hora_fin, descripcion	HORA_GUARDIA
Clase	profesor (FK), hora (FK), aula (FK), fecha, asignatura (FK), aula_profe	CLASE

Vista principal: dashboard_view(request)

Esta vista protegida con @login_required realiza las siguientes operaciones:

- Obtiene el profesor según el email del usuario autenticado
- Consulta todas las clases del profesor con optimización select_related
- Calcula estadísticas:
 - total_guardias: Total de clases asignadas
 - guardias_pendientes: Clases futuras (fecha >= hoy)
 - guardias_aceptadas: Clases pasadas (fecha < hoy)
 - guardias_hoy: Clases programadas para hoy
- Renderiza template con contexto completo

3. CONFIGURACIÓN PRINCIPAL (settings.py)

Parámetro	Valor
Django Version	6.0
Base de datos	MySQL
Host BD	62.175.162.13
Database	GUARDIASDB
User BD	dbadmin
Port	3306
Conector	mysqlclient==2.2.7
Zona horaria	Europe/Madrid
LOGIN_URL	/login/
LOGIN_REDIRECT_URL	/dashboard/
LOGOUT_REDIRECT_URL	/login/

4. FLUJO DE AUTENTICACIÓN Y ACCESO

El sistema implementa un flujo de autenticación estándar de Django:

1. Usuario sin autenticar

- ↓ Accede a /login/
- ↓ Se muestra login_view con formulario

2. Usuario introduce credenciales

- ↓ POST a /login/
- ↓ Validación con AuthenticationForm

3. ¿Credenciales válidas?

- SÍ: login(request, user) → Redirige a /dashboard/
- NO: Muestra error y redisplay del formulario

4. Usuario autenticado accede a /dashboard/

- ↓ @login_required verifica sesión
- ↓ dashboard_view obtiene Profesor por email
- ↓ Se consultan las Clases del profesor
- ↓ Se calculan estadísticas
- ↓ Se renderiza dashboard.html con datos

5. Usuario ve interfaz con sus guardias

5. MODELOS Y RELACIONES DE BASE DE DATOS

El modelo de datos está diseñado con las siguientes relaciones:

Modelo central: Clase

La tabla CLASE actúa como entidad central que relaciona:

- Un Profesor (FK) - Quién imparte
- Una HoraGuardia (FK) - Cuándo se imparte
- Un Aula (FK) - Dónde se imparte
- Una Asignatura (FK) - Qué se imparte
- Una fecha (DateField) - Día específico
- Campo falta_profe (Boolean) - Indica si el profesor ha faltado

Restricción de unicidad:

La combinación (profesor, hora, aula, fecha, asignatura) debe ser única para evitar duplicados en el sistema.

Queries principales utilizadas:

```
# Obtener profesor por email
profesor = Profesor.objects.get(email=request.user.email)

# Obtener guardias optimizadas
Clase.objects.filter(profesor=profesor).select_related(
    'asignatura', 'aula', 'hora'
).order_by('fecha', 'hora__hora_inicio')

# Filtros utilizados
datos.filter(fecha__gte=date.today()) # Futuras
datos.filter(fecha__lt=date.today()) # Pasadas
datos.filter(fecha=date.today()) # Hoy
```

6. DEPENDENCIAS Y REQUISITOS

El archivo **requeriments.txt** especifica las siguientes dependencias:

Paquete	Versión	Descripción
asgiref	3.11.0	Implementación ASGI para Django
Django	6.0	Framework web principal
mysqlclient	2.2.7	Conector MySQL (requiere libmysqlclient)
sqlparse	0.5.4	Parser SQL para debugging

7. RUTAS Y ENDPOINTS

Ruta	Método	Descripción	Auth
/login/	GET/POST	Autenticación de usuarios	No
/logout/	GET	Cierre de sesión	Sí
/home/	GET	Redirige a dashboard	Sí
/dashboard/	GET	Panel principal con guardias	Sí
/admin/	GET/POST	Panel administrativo Django	Sí (superuser)

8. SEGURIDAD Y VALIDACIÓN

Medidas de seguridad implementadas:

- **Autenticación:** Decorador `@login_required` en vistas sensibles que requieren autenticación del usuario.
- **Protección CSRF:** Token CSRF en formularios mediante template tag `{% csrf_token %}` para prevenir ataques Cross-Site Request Forgery.
- **Validación de credenciales:** Django `AuthenticationForm` valida formato y autenticidad de credenciales.
- **Prevención SQL Injection:** Uso del ORM Django que parametriza automáticamente las consultas SQL.
- **Gestión de sesiones:** `SessionMiddleware` gestiona sesiones seguras con cookies firmadas.

Vulnerabilidades potenciales a revisar:

- Credenciales de base de datos hardcodeadas en `settings.py`
- `DEBUG = True` podría estar activo en producción (inseguro)
- `SECRET_KEY` podría estar expuesto en repositorio público
- Considerar implementar HTTPS y configurar `SECURE_SSL_REDIRECT`

9. GUÍA DE DESARROLLO Y EJECUCIÓN

Instalación de dependencias:

```
pip install -r requirements.txt
```

Crear y aplicar migraciones:

```
python manage.py makemigrations  
python manage.py migrate
```

Crear superusuario (admin):

```
python manage.py createsuperuser
```

Ejecutar servidor de desarrollo:

```
python manage.py runserver
```

Acceder a la aplicación:

- Aplicación: <http://localhost:8000/login/>
- Panel admin: <http://localhost:8000/admin/>

10. INFORMACIÓN ADICIONAL Y OBSERVACIONES

Nombre del proyecto: Kronoleon (referencia al tiempo - "Kronos" + "León")

Objetivo: Sistema de gestión de guardias para IES Los Manantiales (Torremolinos)

Equipo de desarrollo:

- @AlejandroMelFer
- @cachispo
- @chopox
- @FatinBnCh
- @MikeSr
- @rubenn16

Estado actual del proyecto:

- ✓ Base de datos MySQL conectada
- ✓ Modelos creados y migraciones generadas
- ✓ Templates modificados y diseñados
- ✓ Sistema de autenticación funcional
- ✓ Dashboard con visualización de guardias
- ✓ Cálculo de estadísticas en tiempo real

Próximas mejoras recomendadas:

1. Registrar modelos en admin.py para gestión desde panel administrativo
2. Implementar funcionalidad para marcar guardias como aceptadas/rechazadas
3. Agregar filtros y búsquedas avanzadas en tablas de guardias
4. Implementar API REST para posible integración con frontend SPA
5. Agregar sistema de notificaciones/alertas para guardias próximas
6. Implementar estadísticas más complejas con gráficos visuales
7. Funcionalidad de exportar reportes a PDF/Excel
8. Sistema de solicitudes de cambio de guardia entre profesores
9. Calendario visual interactivo de guardias
10. Sistema de ausencias y justificaciones

11. CONCLUSIÓN

El proyecto **Kronoleon** representa una solución completa y moderna para la gestión de guardias de profesores en centros educativos. Construido sobre Django 6.0, aprovecha las mejores prácticas del framework para ofrecer un sistema robusto, seguro y escalable.

La arquitectura del proyecto separa claramente las responsabilidades entre la autenticación (app usuarios) y la lógica de negocio principal (app dashboard), facilitando el mantenimiento y futuras extensiones del sistema.

El modelo de datos está bien diseñado con relaciones claras entre las entidades principales (Profesor, Clase, Aula, Asignatura, HoraGuardia), permitiendo consultas eficientes y manteniendo la integridad referencial.

La interfaz de usuario implementada en los templates ofrece una experiencia moderna y responsive, con características como modo oscuro, navegación intuitiva y visualización clara de estadísticas.

El proyecto está en un estado funcional sólido y listo para ser extendido con las funcionalidades adicionales propuestas en la hoja de ruta de mejoras.

Documentación generada el 14/12/2025 a las 16:42